



Software Engineering Department
ORT Braude College

Capstone Project Phase A – 61998

Enhancing Electric Vehicle Routing Efficiency Through K-Means Clustering and Genetic Algorithm Optimization

24-1-R-20

Supervisor: Prof. Miri Weiss-Cohen

Team members: Yovel Gabay, Tal Gaon

Git link: <https://github.com/Yovelgabay/Electric-Vehicle-Routing.git>

Table of Contents

Abstract.....	3
1. Project Review	3
1.1. Introduction	3
1.2. The Algorithm	5
1.3. Research Process	6
1.3.1.Clustering	6
1.3.2.Route Representation	7
1.3.3.Genetic Algorithm for Routing Problem	8
1.4. Experimental Setup and Results	18
1.4.1.An overview of the operator methods being researched.....	19
1.4.2.Comparison of Parent Selection Methods	20
1.4.3.Comparison of mutation methods and rates	21
1.4.4.Comparison of Crossover Methods	22
1.4.5.Impact of Adding Extreme Chromosomes	22
1.4.6.Impact of Best Route Preservation	23
1.4.7.Impact of Passing the Best Route During Dynamic Assessment	24
1.4.8.Conclusions from the Results.....	25
2. User Guide.....	26
2.1. Overview of the System	26
2.2. Key parameters configuration (From parameters.py).....	26
2.3. Visual and execution output:	27
2.4. How the algorithm generates the route and user customization:	27
3. Maintenance Guide	28
3.1. Purpose	28
3.2. Operating Environment	28
3.2.1.Software Requirements	28
3.2.2.Installation Instructions	28
3.3. Flexibility and Customization	29
3.3.1.Flexible Combinations of Methods	29
3.3.2.Adding New Methods	29
4. References	30

Abstract

Efficient route planning for electric vehicles (EVs) presents a critical challenge in the transportation sector, particularly with the rising concerns over greenhouse gas emissions and energy sustainability. This project addresses the challenge of determining the most efficient route from point A to point B, considering stops at charging stations by integrating K-Means clustering and Genetic Algorithm (GA) optimization techniques to enhance routing efficiency. Traditional methods often overlook real-world challenges such as queuing delays at charging stations. Our approach tackles these issues through two main strategies: K-Means clustering to partition charging stations into clusters based on geographical proximity and then Genetic Algorithm optimization to minimize travel distance by strategically selecting routes with minimal charging stops. By integrating these techniques, our approach ensures efficient charging and minimizes overall travel distance, addressing the unique challenges posed by EVs. This research contributes to advancing sustainable transportation solutions, supporting the transition towards a greener and more efficient transportation sector.

Keywords:

Genetic algorithm; Electric Vehicle Routing with Waiting Times at Recharging Stations; Optimal charging station selection; K-means clustering;

1. Project Review

1.1. Introduction

The vehicle routing problem (VRP) poses a significant challenge in optimizing delivery routes from a depot to scattered customers while maintaining various constraints [1]. Dantzig and Ramser defined the problem for the first time in 1959 as the Truck Dispatching Problem [2]. VRP is a generalization of the well-known traveling salesman problem (TSP) that seeks the shortest route to visit all customers. VRP encompasses various problem variants like capacitated VRP (CVRP), VRP with time windows (VRPTW), and multidepot VRP (MDVRP), each tailored to specific real-world scenarios and constraints. [3]

In the context of modern transportation, The transportation sector faces a critical crossroads, with conventional vehicles contributing 75% of its greenhouse gas emissions (IEA, 2022) [4] This alarming reality concerns climate change, resource depletion, and the future of our planet. While electric vehicles (EVs) have emerged as a potential solution, their environmental impact remains under examination. Although they are known for being emissions-free during operation, their dependence on electricity, often generated from fossil fuels, raises questions about their overall environmental benefit. Their rapid adoption could also strain energy grids, further complicating the equation. Despite these challenges, EVs present undeniable advantages. Government incentives

and regulations actively encourage their use, driving a rapidly growing market with a 72% annual average increase in global stock from 2017 to 2021 (IEA, 2022) [4]. However, the environmental benefits of EVs are contingent on efficient route planning and utilization.

This introduces the Electric Vehicle Routing Problem (EVRP), an extension of VRP tailored to the unique requirements of EVs [5]. EVRP aims to optimize routes for electric vehicles while considering charging constraints, energy consumption, and other specific factors like travel distance, time, and charging station logistics.

In our project, we focus on the main objective: Incorporating Real-World Charging Constraints: Many studies often assume that recharging happens immediately when EVs arrive at charging stations. However, real-world scenarios often involve limited charger availability, resulting in queuing before recharging. This waiting time can significantly impact routing decisions.[6] We aim to provide more realistic and practical routing solutions by integrating queuing times into our model. To simulate this objective, we use clustering techniques to group charging stations based on their proximity. This allows the algorithm to dynamically adjust routes based on current queuing times, optimizing for both travel distance and time spent waiting for available chargers.

Based on the EVRP problem, our problem is described as follows: Given a start point A and destination point B, **each EV can complete the journey while scheduling the best routes within its battery charge level limits.** This entails strategically planning charging stops along the route to recharge the EVs' batteries. Finding the best route depends on stopping at the most efficient charging stations, including considering:

1. Closest charging station
2. The queuing times at charging stations

Our approach involves two phases:

1. Partitioning Charging Stations into Clusters: We will use the K-Means clustering algorithm to divide the charging stations into clusters based on geographical proximity. Figure 1 provides a visual representation of the clustering of recharging stations. Each cluster will have known waiting times for its charging stations. We will assume an average waiting time for stations with unknown waiting times. At any point along the route, we can access waiting times exclusively for the charging stations within the current cluster.
2. Optimizing the Total Travel Distance: In the second phase, we aim to optimize the total travel distance by determining the best charging stations for the EV to stop along the route. This will be achieved using a Genetic Algorithm (GA) to strategically select the route with charging stations, considering factors such as the distance to the station and the potential queue of cars waiting for recharging.

Whenever a driver enters a new cluster, our model will be re-evaluated to find a new optimal route, considering the known waiting times of the nearby stations. This dynamic reassessment ensures that the driver always follows the most efficient route based on the information on the latest charging station waiting times.

1.2. The Algorithm

In this study, we address the challenge of optimizing electric vehicle (EV) routes to minimize travel distance and waiting times at charging stations while ensuring the EV's battery capacity is not exceeded. The complexity arises from dynamically navigating through multiple scattered charging stations with varying queuing times. To simulate real-time conditions, we first apply the K-means clustering algorithm to group charging stations based on their geographic locations. This approach models a scenario where real queuing times are only known for nearby stations, adding a layer of realism to the routing problem.

Once the charging stations are clustered, we use a Genetic Algorithm (GA) to dynamically determine the best charging station at each step. The GA iteratively applies genetic operators, such as parent selection, crossover, and mutation, to generate and improve potential solutions, to minimize total distance and waiting time. Importantly, the algorithm is dynamic: instead of planning the entire route, it recalculates the best path at each step based on the vehicle's current position, battery level, and updated queuing times at charging stations. This ensures that the route remains flexible and optimized in real-time, adjusting to the vehicle's evolving location. By continuously adapting the route as the EV moves, our approach provides a practical and realistic solution to the challenges of EV navigation.

Pseudo Code:

1. **Generate route and random charging stations with queueing times.**
Purpose: Simulate a route for the EV, including potential charging stations and queueing times.
2. **Get connections between route and charging stations**
Purpose: Associate each charging station with a specific route point that serves as an exit point to the associated charging station.
3. **Apply K-means clustering to the charging stations**
Purpose: Cluster charging stations by proximity to allow dynamic route adjustments based on queueing times.
 1. Perform K-means clustering on charging stations.
 2. Assign each route point to the nearest centroid.
4. **Dynamic optimization process using a Genetic Algorithm**
Purpose: Reassess the route and find the best charging stations for the current EV position based on updated queueing times at charging stations, optimizing both travel distance and waiting time.
 1. Iterate over each point on the route as a starting point.
 2. For each starting point:

1. Update the route according to the EV position.
2. Update relevant data based on the new starting point.
 - Adjust EV capacity based on the current position.
 - Retrieve updated connections between route and charging stations.
 - Remove irrelevant stations.
3. Run the genetic algorithm to determine the best charging stations for the current route.
4. If a charging station from the current solution is located at the start point:
 - Recharge the EV.

1.3. Research Process

1.3.1. Clustering

In our project, we initially wanted to simplify the problem and enhance the realism of dynamic route evaluations by clustering the charging stations. We assigned each route point to the nearest cluster of charging stations and ran a genetic algorithm to evaluate the optimal stops. The approach was designed so that if a charging station belonged to the same cluster as the current route point, we would use the actual queueing time at that station. Conversely, if the charging station was in a different cluster, we would assume an average waiting time. This method aimed to improve the realism of our model by acknowledging that real queueing times are probably known when a route point is close to the charging station. Figure 1 shows a division of all the stations scattered along the route into clusters.

We planned to determine the best number of clusters, K , using the silhouette score. However, during development, we realized that clustering did not significantly reduce complexity and that the silhouette score suggested too few clusters. This limitation would not sufficiently highlight the dynamic capabilities of our system. As a result, we adjusted our approach and chose K to be the number of route points divided by 3. This adjustment effectively demonstrated the dynamic features of our system while maintaining a manageable level of complexity.

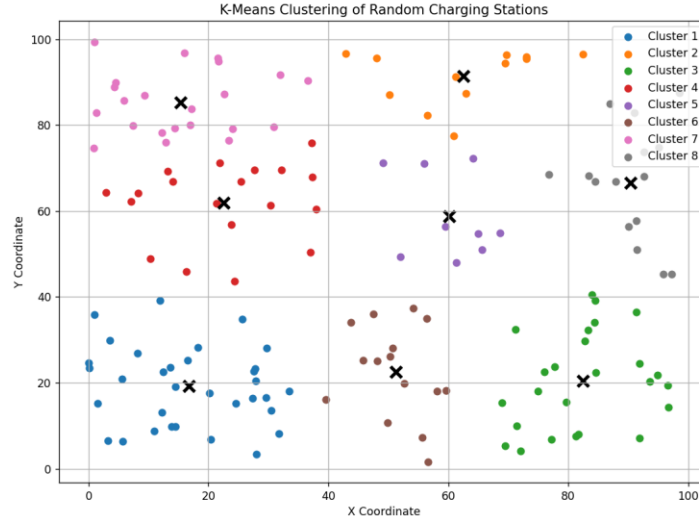


Fig. 1. K-means clustering of charging stations along the route.

1.3.2. Route Representation

In our algorithm, the route is represented as a sequence of points defined by their (x, y) coordinates. The route connects a starting point, several checkpoints, and an ending point, allowing it to mimic a realistic path. Importantly, these points not only define the route but also act as exit points to the charging stations. These exit points are essentially junctions where vehicles can leave the main route to access nearby charging stations and then return to the route after charging. An example of the route is shown in Figure 2. Additionally, each route point is assigned to the cluster whose centroid is closest to it. This means that when a vehicle reaches a specific route point, it is within the cluster that the route point is assigned to.

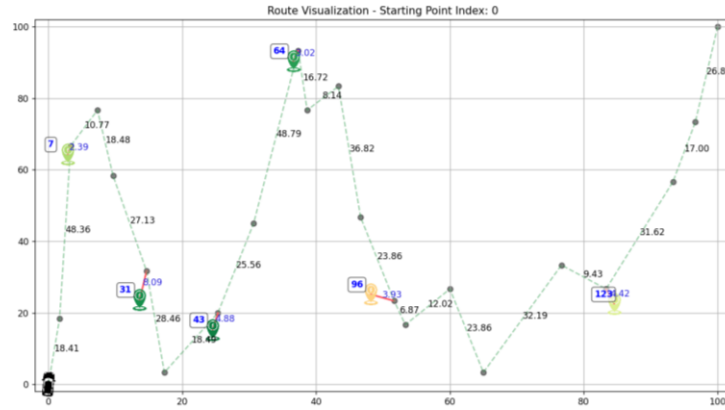


Fig. 2. Route representation with exit points to the charging stations.

Charging Stations

To efficiently manage the charging stations along the route, we first generate random points representing charging station locations. Each station is assigned a random penalty

representing queuing times, varying from 0 to 20 minutes. Then, we calculate the Euclidean distance between each station and every point on the route to map the station to the closest route point, which serves as the station's exit point.

This mapping has two key implications:

1. If the algorithm selects a charging station to stop at, it can only be accessed via its corresponding route point, which serves as the exit point to the station.
2. Each route point can lead to a different number of charging stations. All stations accessible from the same route point are sorted in the same order, maintaining their sequence in the proximity-based sorting. This sorting is crucial for the subsequent genetic algorithm process.

1.3.3. Genetic Algorithm for Routing Problem

Introduction to Genetic Algorithm:

Genetic algorithms (GAs) are a powerful tool inspired by the process of natural selection, first introduced in 1970 by John Holland. They have proven to be particularly effective at solving complex optimization problems. Much like nature's evolution, the GA process begins with selecting individuals (chromosomes) for the first generation, manipulating them through crossover and mutation techniques to generate subsequent generations. The idea is that if parents have better fitness, their offspring will be better than the parents and have a better chance of surviving. Through an iterative process, the algorithm continues refining the population until it converges on the best individual, representing the optimal or near-optimal solution for the routing challenge at hand. [11]

Pseudo Code for Genetic Algorithm [11]

1. **Initialization**- Generate n chromosomes for the initial population p
2. **Loop** until the end condition is met
 - a. **Evaluation**- Evaluate the fitness of each chromosome in p
 - b. **While** n offspring have not been created, **do**
 - i. **Selection**- Select 2 parents from P based on their fitness
 - ii. **Crossover**- Perform crossover to create offspring. If no crossover was performed, the offspring is an exact copy of the parents
 - iii. **Mutation**- With a mutation probability, mutate new offspring
 - iv. **Accepting**- Place new offspring in a new population
 - c. **Replace**- Replace old population with the new population
3. **Return** the best solution

The genetic algorithm (GA) is a search heuristic that mimics natural selection. It is widely used to generate high-quality solutions for optimization and search problems. Here, we explain the key components and processes of our GA [7].

Genetic Representation

In our GA, a chromosome represents a possible solution to the routing problem. Each chromosome consists of sequences of positive integers, where each integer represents the ID of a charging station through which a route passes. The chromosome length is variable and can range from a single node to the total number of nodes in the network [7]. This variable-length chromosome allows the algorithm to explore different routing paths adaptively.

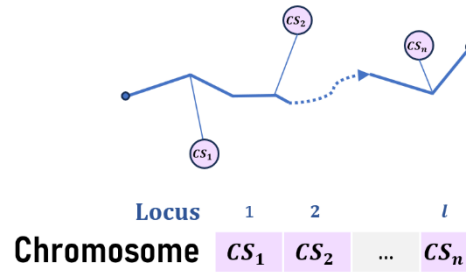


Fig. 3. Chromosome representation.

Population Initialization

The initial population of chromosomes is generated randomly, with each chromosome representing a possible route through selected charging stations along the route. The number of charging stations in each chromosome can vary, ensuring diversity in the initial population. This diversity is crucial for the genetic algorithm (GA) to explore various possible solutions within the search space.

Our `initialize_population()` function seeds the algorithm with an initial set of possible routes, including the shortest and longest possible paths. This genetic diversity is vital to avoid converging on suboptimal solutions prematurely. The initial population includes:

1. **Shortest Valid Path:** A chromosome representing the shortest valid path with at least one intermediate stop.
2. **Longest Path:** A chromosome representing the longest path that goes through all charging stations.
3. **Random Paths:** The other chromosomes represent random paths with varying numbers of stops between the origin and destination.

This mix of initial solutions helps ensure that the genetic algorithm has a diverse starting point, which is essential for exploring the solution space effectively and avoiding premature convergence.

```

num_points = len(points_with_ids)

# Add a chromosome with a single randomly chosen charging station (shortest
valid path)
if num_points > 0:
    shortest_path_chromosome = [random.choice(range(num_points))]
    population.append(shortest_path_chromosome)

# Add a chromosome with a path covering all charging stations (longest path)
longest_path_chromosome = list(range(num_points))
population.append(longest_path_chromosome)

```

Code snippet. 1. Adding the longest route and shortest route to the population.

Fitness Function

The fitness function is a key component in genetic algorithms, determining how well a solution (chromosome) satisfies the problem requirements. It guides the selection process by evaluating and assigning a fitness score to each chromosome. In the context of the electric vehicle (EV) routing problem, the fitness function assesses how well a route minimizes distance and waiting time at the charging station while ensuring that the EV's capacity constraints are not violated.

Here is a detailed breakdown of the fitness function and how it contributes to optimizing EV routing:

1. Validity and Exceeded Kilometers: The function first checks if the route (chromosome) is valid by ensuring that the distance between consecutive charging stations (the distance refers to the route segment that connects two consecutive charging stations) does not exceed the EV's capacity. If the route is invalid, it calculates the excess kilometers that exceed the EV's range, applying a substantial penalty to discourage infeasible routes. As part of our study, we added a penalty factor to this calculation. We found that without multiplying the exceeded kilometers in the penalty factor, there was no significant impact on the results, and illegal chromosomes were still being selected. To address this, we researched and selected an appropriate penalty factor, ensuring that infeasible routes are heavily penalized and thus less likely to be chosen. In addition, we chose not to disqualify illegal chromosomes outright. Instead, we prioritized those with higher fitness among the illegal ones. This approach allows us to retain potentially useful routes, which, through future crossovers and mutations, can evolve into better, shorter, and legal routes.
2. Total Distance penalty: The function computes the route's total distance by summing the distances from each stop along the route. The goal is to minimize this distance to reduce travel time and energy consumption.
3. Waiting time penalty: Each stop incurs a penalty based on the waiting time at the charging station, with each waiting time multiplied by a queueing time penalty factor, which increases the weight of queueing time on the overall fitness score. This factor ensures that the genetic algorithm considers routes with shorter wait times. For stops within the current cluster (i.e., charging stations close to the EV's

current location), the penalty is based on the actual waiting time, simulated by drawing random waiting times to reflect real-world conditions. An average waiting time is used for all other stations outside the current cluster. This distinction ensures that the fitness function accurately reflects expected waiting times and prioritizes stops with shorter wait times, contributing to a more efficient route.

4. **Fitness Calculation:** The fitness score is the inverse of the sum of total distance, penalties, and exceeded kilometers, ensuring that shorter, less penalized routes are ranked higher.

```
# Compute the total penalty
total_penalty = total_distance + queueing_time_penalty + exceeded_km

# Return the inverse of the total penalty to represent fitness & handle
division by zero
return 1 / total_penalty if total_penalty > 0 else 1e-12
```

Code snippet. 2. Calculate of chromosome's fitness function.

By optimizing these parameters, the fitness function ensures that the genetic algorithm evolves efficient routes, complies with EV capacity constraints, and minimizes travel time and penalties. This results in a robust solution for the EV routing problem, enhancing the overall efficiency of the electric vehicle network.

Tournament Selection

Tournament selection is used to select parents for the crossover operation. In this method, a subset of chromosomes is randomly chosen from the population, and the chromosome with the highest fitness within this subset is selected as a parent. This process is repeated to select the second parent. The subset size (k) significantly impacts the efficiency of the algorithm; hence, choosing an appropriate value for k is crucial, as it affects both the selection pressure and the balance between exploration and exploitation. A well-chosen k value ensures that fitter chromosomes have a higher chance of being selected while still maintaining diversity within the population, which is essential for the genetic algorithm's exploratory capabilities.

Variant Crossover

Crossover is a genetic operator used to combine the genetic information of two parents to generate new offspring. In the context of our routing problem, crossover plays the role of exchanging partial routes between two chosen chromosomes. The crossover process can be divided into two scenarios:

Crossover Process with Common Nodes: When the parent chromosomes share common nodes, the crossover points are determined by one of these common nodes randomly, and the segments before and after this point are swapped between the parents to create two new offspring.

Crossover Process with No Common Nodes: Instead of selecting a crossing node from

common nodes, the crossover is performed at a point that splits both parents into roughly equal segments. This point is typically chosen as the midpoint of the parent routes. The segments before this crossover point in one parent are combined with the segments after the crossover point in the other parent, and vice versa, to create the offspring. This is followed by sorting the chromosomes to ensure that the chromosomes are valid.

Given the unique structure of our problem and route representation, standard crossover methods were insufficient. We developed a custom crossover tailored to our needs, enabling the exploration of new routes while preserving successful segments from parent routes for better adaptability to the problem's constraints.

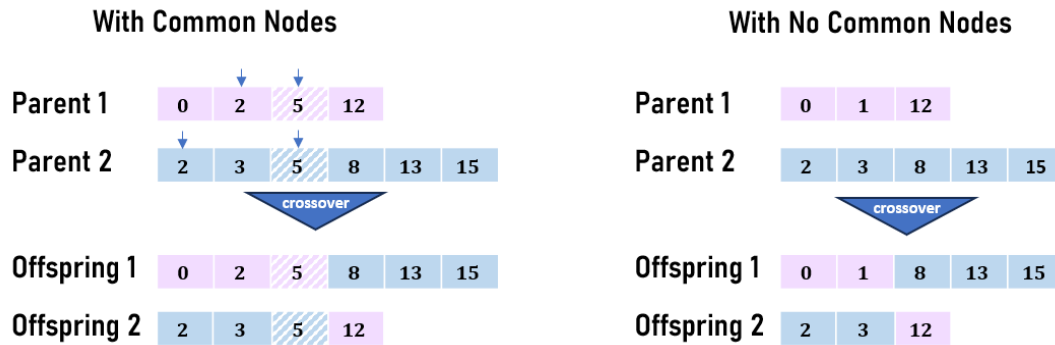


Fig. 4. Crossover process with and with no common nodes.

Variant Mutation

The mutation is another genetic operator used to maintain genetic diversity within the population. It involves making small random changes to a chromosome. This process helps to avoid getting stuck in local optima. In addition, the algorithm tries to ensure that there are always new and diverse routes in the population, which helps in finding a more optimal solution.

Our mutate function explores new routes by:

- Replacing a single station
- Adding a new station
- Removing a single station

Practical Implications

- For short routes: (length 1 or 2), the mutations are simpler (replacing or adding stations) to quickly explore different configurations.
- For longer routes: the mutations are more complex (removing or adding) to make more significant changes and explore a wider range of possibilities.

Given the nature of our routing problem and route representation, standard mutation methods were insufficient. We developed a custom mutation approach to ensure diversity

and effectiveness, maintaining a balanced population crucial for the genetic algorithm's success in finding optimal EV routing solutions.

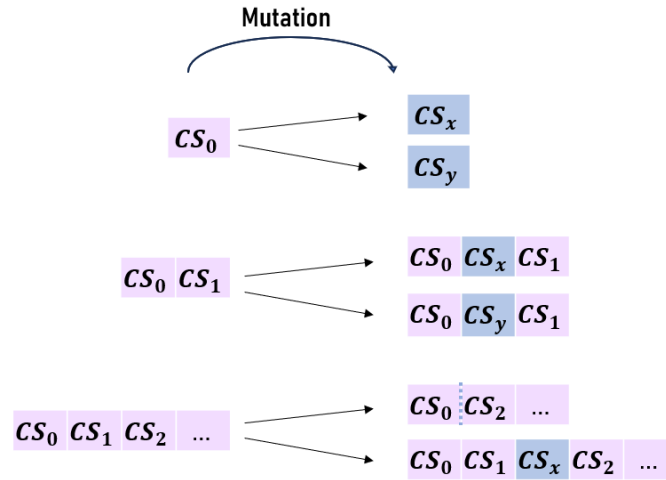


Fig. 5. Mutation operation in our Genetic Algorithm.

Stagnation

Stagnation refers to a situation in genetic algorithms where there is little or no improvement in the solution quality over successive generations. This often indicates that the population has converged to a local optimum or lacks the genetic diversity necessary to produce better offspring. Recognizing stagnation is crucial for the efficient use of computational resources and often triggers the termination of the algorithm. By setting a threshold for stagnation, we can prevent the algorithm from running indefinitely when no improvement is being made, thereby saving computational resources and time.

The termination of a genetic algorithm is a critical aspect of its design. It can be determined by various criteria, such as reaching a maximum number of generations or encountering a prolonged period of stagnation, where no improvement is observed.

```
if current_best_fitness > best_fitness:
    best_fitness = current_best_fitness
    best_route = evaluated_population[0]
    stagnation_counter = 0 # Reset stagnation counter since improvement occurred
else:
    stagnation_counter += 1 # Increment stagnation counter if no improvement

# Stop if there has been no improvement for the defined number of generations
if stagnation_counter >= max_stagnation:
    break
```

Code snippet. 3. Stagnation code.

Dynamic process

The dynamic process in our algorithm simulates the real-time calculation of an updated route for an electric vehicle (EV) as it travels. At each step, we shorten the portion of the route that the EV has already traversed, from the beginning of the route to the current point, which reflects the EV's current position. This ensures that only the relevant segments of the route and associated charging stations are considered in each iteration.

Here's a detailed breakdown of what happens with each route adjustment:

1. Cluster Assignment: Each route point is assigned to the nearest cluster, so when evaluating a specific route point, the algorithm only considers the real queuing times of the charging stations within the same cluster as that route point.
2. Charging Station Removal: When the algorithm removes a route point that has already been traversed, it also removes only the charging stations that are no longer reachable from the remaining route, ensuring that only the relevant stations connected to the remaining route points are kept in the solution.
3. Updating Queuing Times: The algorithm updates the relevant queuing times at the charging stations based on the cluster corresponding to the EV's current location. This update ensures that the queuing times at the nearest charging stations reflect the real-time situation.
4. Adjusting EV Capacity: The algorithm also adjusts the EV's initial capacity to account for the distance already traveled, specifically considering the distance from the last charging station where the EV recharged to the starting point of the shortened route. This adjustment is crucial for accurately planning the next charging stop, considering the remaining battery capacity.
5. Passing the Best Route to the Next Segment: After each recalculation, the best route for the current segment is passed forward to the next segment. This ensures continuity, as the best charging stations from the current step are considered for the next population in the updated route. If a station that was deemed optimal in the best route is no longer available in the subsequent segment, it is removed from consideration before the route is passed on. This process helps maintain the relevance of the charging station data and ensures that only feasible stations are included in the evolving route.

After each of these adjustments, the genetic algorithm recalculates the optimal route for the newly defined (shortened) route, incorporating the updated queuing times at charging stations, which means that at each new segment of the route, the genetic algorithm is restarted until the EV arriving to the destination point.

Ultimately, a station is "chosen" for recharge only if it is on the EV's current position i.e. at the starting point of the shortened route. The idea behind this selection is that the algorithm determines that the EV should charge there based on the updated queuing times. This decision-making process ensures that the algorithm dynamically adapts to the vehicle's real-time situation and the updated waiting times at the charging stations, optimizing the route with each update.

This approach reflects a real-world process, dynamically recalculating the optimal route with each update to ensure the most efficient path is chosen based on current conditions.

Visualization of the dynamic process

Figure 6 provides a detailed visualization of an electric vehicle (EV)'s route through various charging stations on a two-dimensional plane. The points in the figure represent charging stations along the route. Charging stations within the EV's current cluster are color-coded according to their real queueing times, as indicated by the color bar on the right. Greenish points represent stations with shorter queueing times, while reddish points indicate stations with longer waiting times.

Charging stations outside the EV's cluster and farther from the vehicle are assigned a fixed color representing the average queueing time. This simulates the uncertainty in predicting precise waiting times at these distant stations, as their exact conditions are not known in real time. Therefore, an estimated average waiting time is used for stations outside the immediate range of the EV.

The numbered boxes (e.g., stations 23, 43, 72, 103, 123) highlight the charging stations chosen for the EV to stop and recharge at the first step of the route planning process. These selections are dynamic and may change as the route is updated during the optimization process.

The information bar at the top provides key parameters of the genetic algorithm used to compute the route, including EV Capacity (100), Population Size (1000), Generations (100), Mutation Rate (1), Number of Charging Stations (150), Average Queueing Time (12), and Max Stagnation (20).

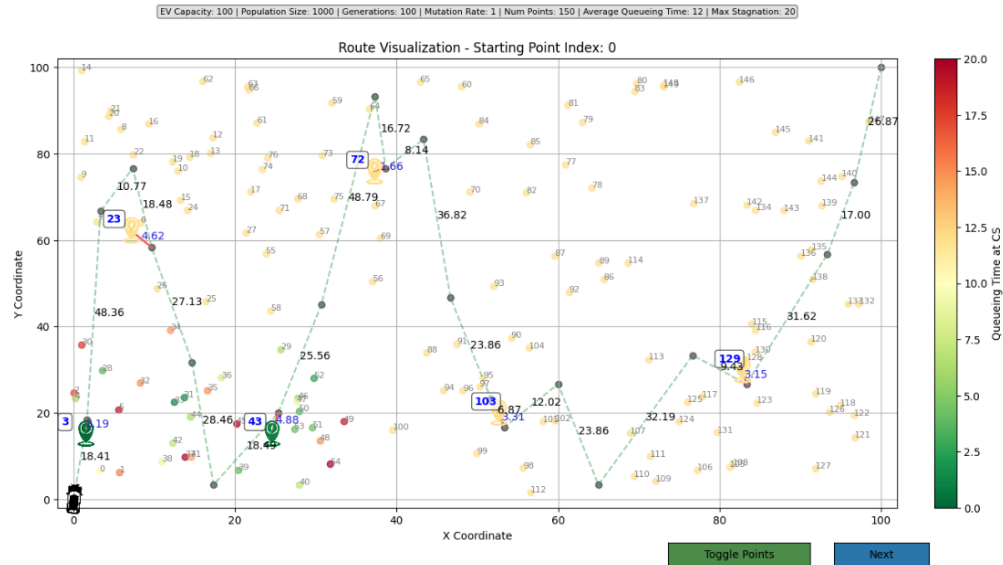


Fig. 6. A route with a charging station along it.

After the EV has traversed the first segment of the route, Figure 7 shows that the charging stations corresponding to the previous section have been removed. Charging Station 3 from the previous index (Starting Point Index 0) was not selected. Instead, due to the dynamic adjustments, the nearest charging station was updated to CS 7, which is farther along the route from the starting point.

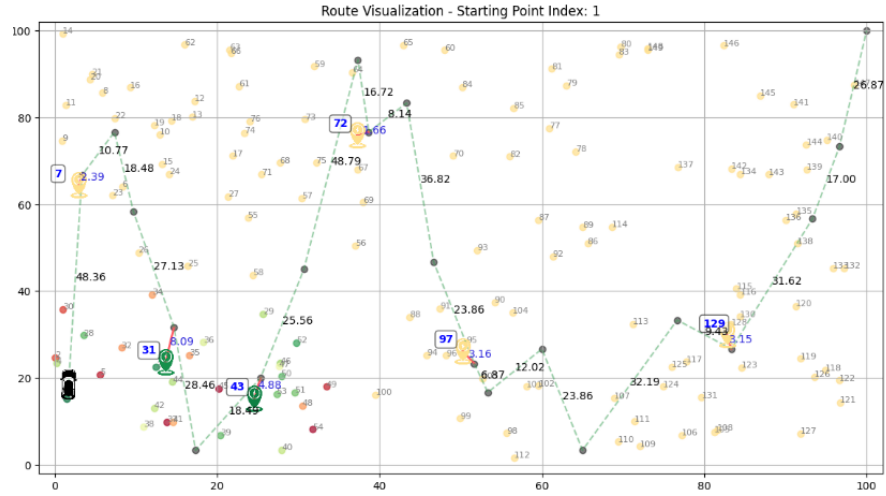


Fig. 7. A route from the starting point index: 1.

In the updated route in Figure 8, the algorithm dynamically skips CS 7. Instead, it selects CS 22, which is farther from the beginning of the route. This change likely optimizes the overall travel distance for the electric vehicle (EV).



Fig. 8. A route from the starting point index: 2.

In Figure 9, the blue marker indicates that the EV has reached a point in the route where a charging station has been selected for charging.

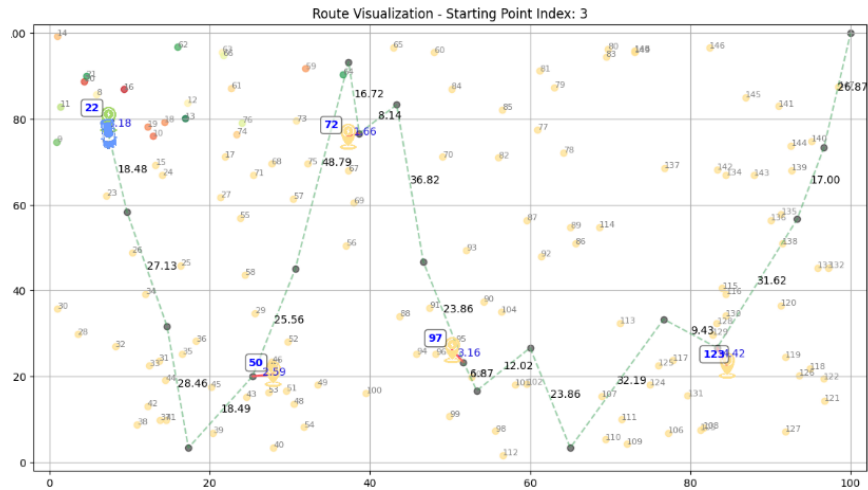
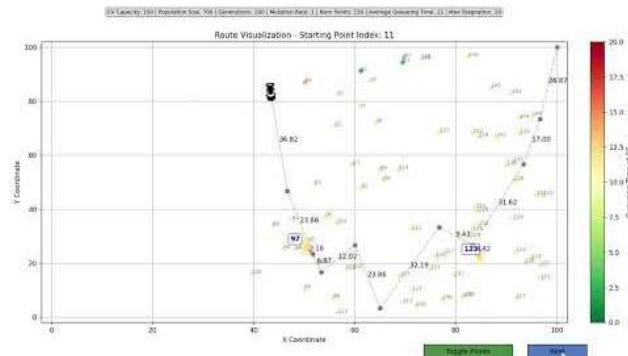


Fig. 9. A route from the starting point index: 3.

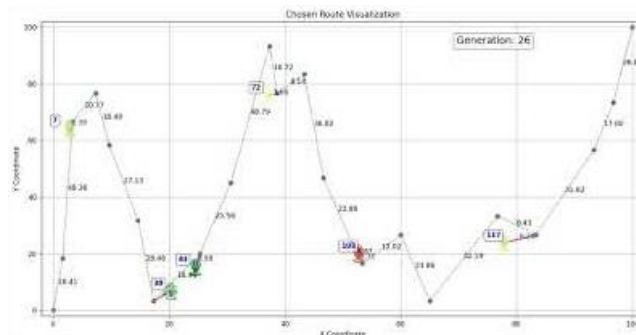
Visualization video of a dynamic route

This video shows how the selection of charging stations changes based on updated waiting times. You can observe the dynamic adjustments in station selection as the waiting times at each station are updated.



Visualization of the best chromosome in each generation (non-dynamic)

This video shows how the selection of charging stations improves across generations as the population evolves. The generation number is displayed in the label above.



1.4. Experimental Setup and Results

1.4.1 Experimental Setup

Several experiments were conducted using simulated datasets to assess the performance of the Dynamic Electric Vehicle Routing (EVRP) algorithm. The datasets simulated real-world scenarios where electric vehicles (EVs) needed to choose optimal routes, considering both travel distance and waiting times at charging stations. The experiments focused on the following parameters:

- **Datasets:** Routes with 150, 300, and 500 randomly placed charging stations were used. Each station was assigned a queuing time between 0 and 20 minutes to simulate real-world congestion.
- **Simulated Queuing Times:** In each experiment, queuing times at charging stations were simulated using a random value generator, constrained between a minimum and maximum waiting time (set to 0 and 20 minutes, respectively). These times were used to model the real-world variability of waiting times at charging stations, which can be influenced by the number of EVs currently charging, station capacity, and charging speeds.
- **Real-World Relevance:** The experiments reflect real-world conditions where charging stations may be limited, and queuing times vary. This setup highlights the need for efficient EV route planning.
- **Algorithm Parameters:** The key parameters of the genetic algorithm (GA) included the following:
 - **Population Size:** Sizes of 200, 500, 1000, and 2000 were tested to explore the balance between solution quality and computation time.
 - **Generations:** 100 generations were used as the baseline, providing ample opportunity for the algorithm to evolve and optimize routes through crossover, mutation, and selection.
 - **Mutation Rate:** 1 (adjusted in some experiments)
 - **EV Capacity:** 100
 - **Max Stagnation:** 20 (i.e., the algorithm terminates if there is no improvement after 20 generations)
- **Objective:** The goal of each experiment was to minimize the total travel distance while factoring in the waiting times at charging stations. In addition to evaluating the performance of different selection methods, mutation techniques, and crossover strategies, we also explored other techniques to further reduce the total travel distance and improve route optimization for EVs.

1.4.1. An overview of the operator methods being researched

In our research, we conducted a comprehensive study to evaluate the impact of different selection, mutation, and crossover methods on the performance of a genetic algorithm designed for optimizing EV routes. Additionally, we explored techniques to improve the algorithm's efficiency.

Selection Methods:

1. **Tournament Selection:** Tournament selection within GAs entails organizing a contest among a predetermined number of individuals. The winner of each tournament, identified as the individual with the highest fitness among the competitors, is then chosen to join the mating pool. By adjusting the tournament size, the level of selection pressure can be modified: larger tournament sizes intensify selection pressure, thereby favoring individuals with superior fitness. This approach effectively facilitates the selection of high-quality individuals for reproduction, thereby enhancing the overall fitness of subsequent generations. However, balancing the tournament size is essential to avoid either slow convergence or premature convergence to suboptimal solutions. Adjusting the tournament size enables fine-tuning the selection pressure to optimize the GA's performance in identifying optimal or near-optimal solutions [8].
2. **Roulette Wheel Selection:** Parents are chosen based on their fitness within the population. Essentially, the fitter the chromosome, the greater its likelihood of being selected. This selection method is analogous to a roulette wheel, where each chromosome occupies a portion of the wheel corresponding to its fitness [9].
3. **Rank Selection:** In situations where there's a significant disparity in fitness levels among chromosomes, for instance, in the traditional roulette wheel selection, if the highest fitness chromosome occupies 90% of the roulette wheel, others have minimal chances of selection. The Rank selection addresses this issue by initially sorting the population based on fitness, assigning each chromosome a rank corresponding to its position in the sorted list. The worst-ranked chromosome receives a fitness score of 1, the second worst 2, and so on, with the best-ranked receiving a fitness score equal to the population size. Subsequently, all chromosomes have an opportunity for selection, with the probability of selection being proportional to their rank rather than their fitness. However, this approach may lead to slower convergence as the differences between the best and other chromosomes are less pronounced [10].

Mutation Methods:

1. **Variant Mutation:** This mutation method, detailed in section 1.3.3 (Genetic Algorithm for Routing Problem)
2. **Random Resetting Mutation-** Randomly altering a single stop in the route, ensuring that the new route is valid by sorting and making sure that isn't a duplicate of existing stops. This method is straightforward but may not always introduce significant variation in the population.

Crossover Methods:

1. **Variant Crossover:** This crossover method detailed in section 1.3.3.
2. **Double-Point Crossover:** The double-point crossover has two crossover points, and the chromosomes between the points are swapped only [10].
3. **Uniform Crossover:** Uniform crossover involves combining bits randomly sampled from both parents. A crossover mask is generated as a random bit string, with each bit independently chosen randomly [10].

These methods were systematically compared to assess their effectiveness in generating optimized routes for the EVs, as detailed in subsequent sections.

1.4.2. Comparison of Parent Selection Methods

The graphs depicted in Figures 10 and 11 show that *Tournament Selection* consistently outperforms both the *Roulette Wheel* and *Rank Selection* across various population sizes and different numbers of charging station points. *Tournament Selection*, which selects parents through competition within smaller subsets of the population, performed particularly well with smaller tournament sizes (e.g., $k=4$). In contrast, the *Roulette Wheel* and *Rank Selection* methods yielded lower fitness scores, especially in more complex scenarios (e.g., with 300 points). This suggests that as problem complexity increases (That is when the number of charging stations is higher), their ability to effectively explore and exploit the solution space diminishes. For simpler scenarios (e.g., with 150 points), there were no significant differences between the parent selection methods, except in smaller populations where *Tournament Selection* showed a slight edge. However, as complexity increased *Tournament Selection* ($k=4$) demonstrated clear superiority and greater stability across most population sizes.

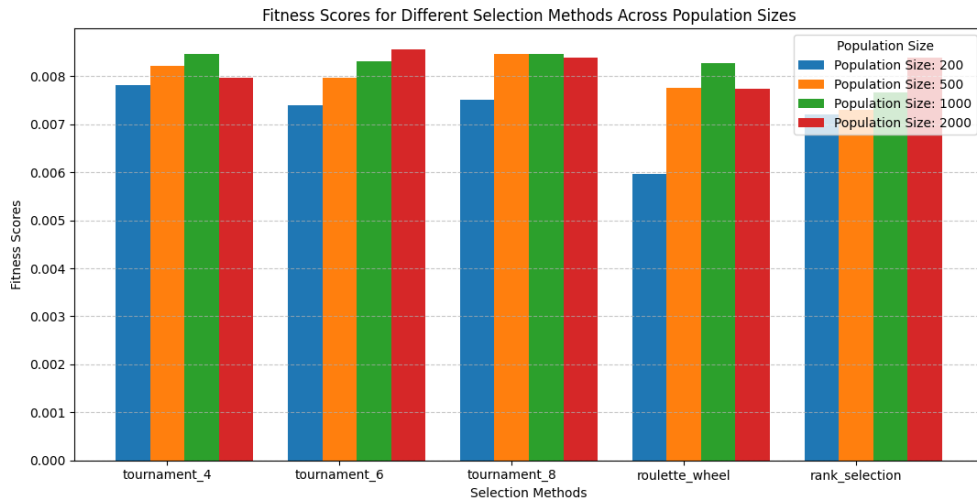


Fig. 10. Fitness score for different selection methods with 150 points (charging stations).

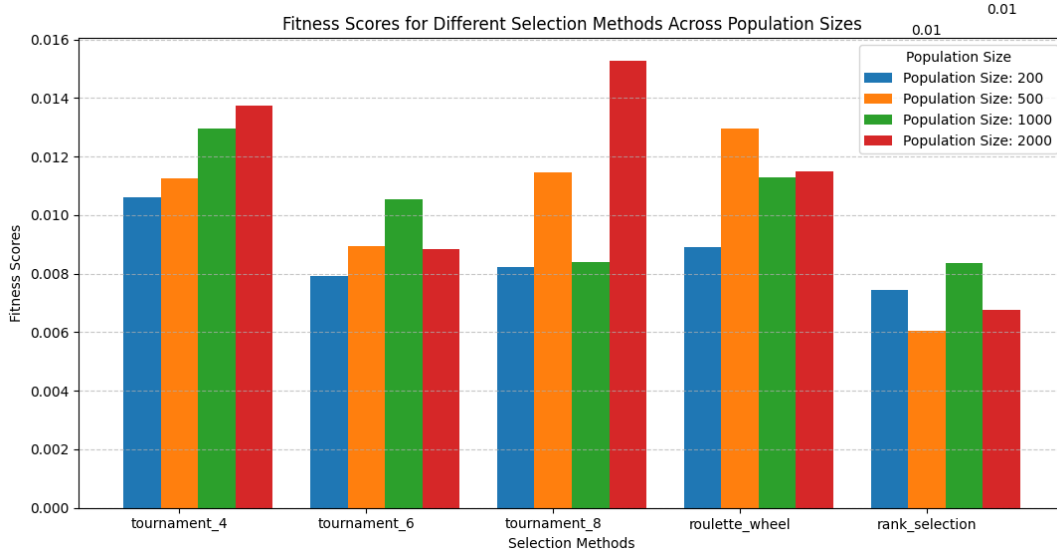


Fig. 11. Fitness scores of the best route for different selection methods with 300 points (charging stations).

1.4.3. Comparison of mutation methods and rates

Figure 12 presents a graph illustrating the fitness scores of the best routes obtained from testing two different mutation methods: the *variant mutation* and the *random resetting mutation*. In this experiment, we varied the mutation rates from 0 to 1 in increments of 0.1, using a population size of 1,000 and 500 charging station points to increase the complexity.

The results show that the *variant mutation* method consistently outperformed the *random resetting mutation*. This difference in performance is especially pronounced at higher mutation rates, where the *variant mutation* method achieves significantly better fitness scores. In contrast, the *random resetting mutation* displayed unstable performance across the range of mutation rates, making it challenging to identify an optimal rate for this method.

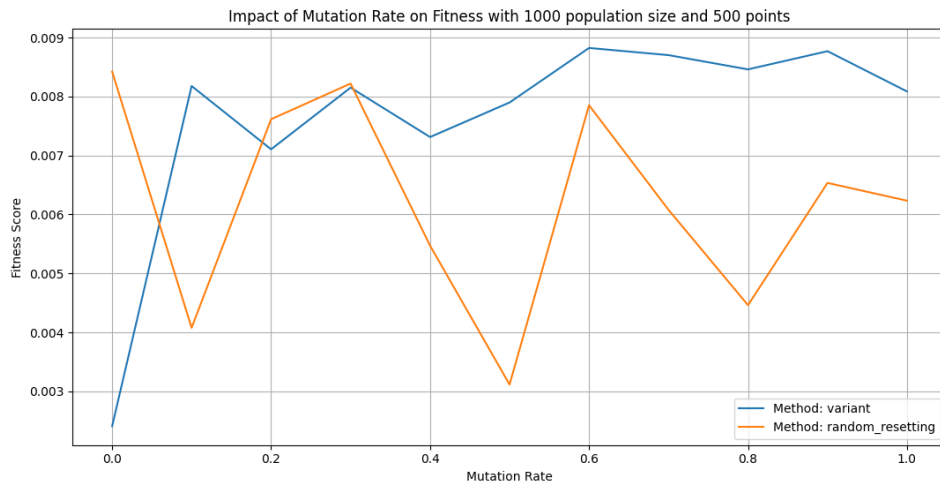


Fig. 12. Fitness scores of the best route for different mutation rate.

1.4.4. Comparison of Crossover Methods

As shown in Figure 13, the *variant crossover* method consistently outperformed both the *double-point* and *uniform* methods across different population sizes. While the *double-point* and *uniform* methods performed similarly, the *double-point* method had a slight advantage in some cases. This suggests that the *variant crossover* is better suited for this specific optimization problem.

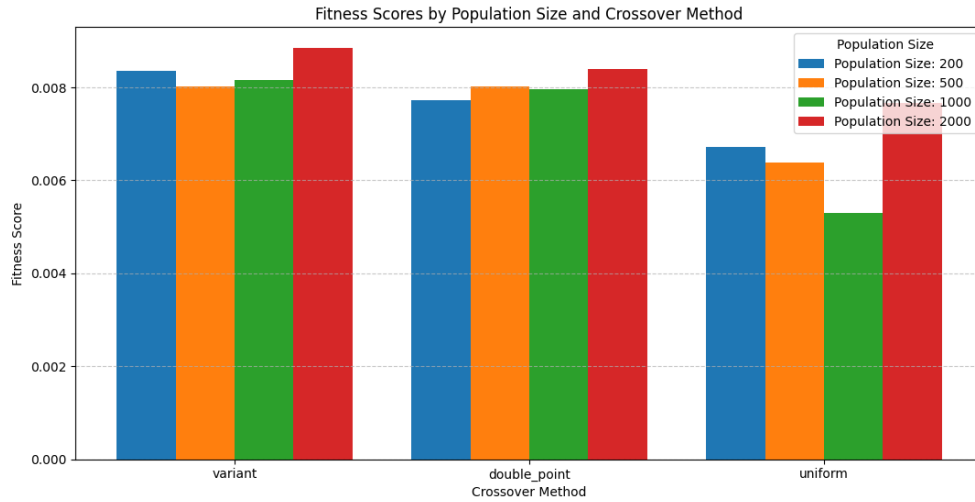


Fig. 13. Fitness score of the best route for different crossover methods.

1.4.5. Impact of Adding Extreme Chromosomes

Adding extreme chromosomes (i.e., the shortest and longest routes) had minimal impact on algorithm performance with 150 charging station points. At this lower complexity level, the additional diversity introduced by extreme chromosomes did not significantly affect the search process. However, with an increase to 300 charging station points, the benefits of including extreme chromosomes became more pronounced. At this higher complexity, most population sizes exhibited improved performance, indicating that the added diversity is crucial for enhancing the algorithm's effectiveness in more complex scenarios.

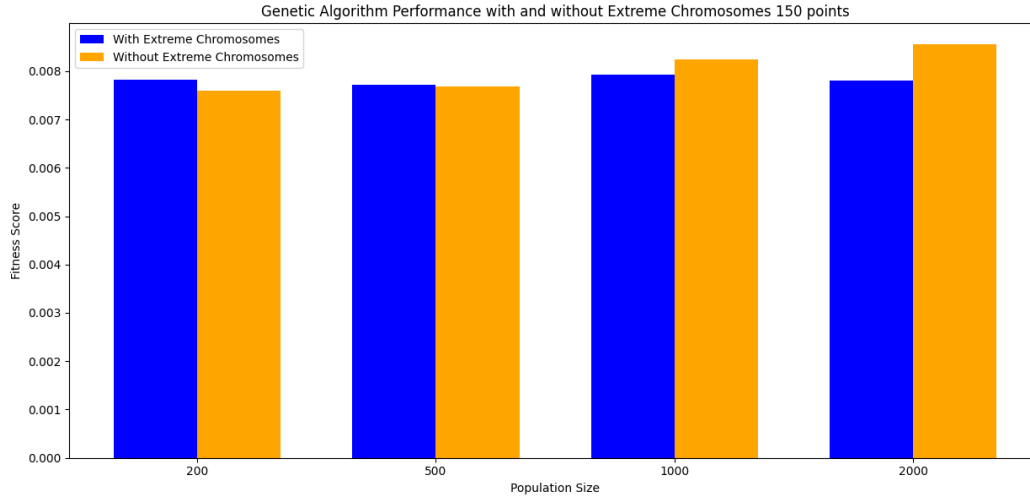


Fig. 14. Fitness score of the best route for different mutation methods with 150 points (charging stations).

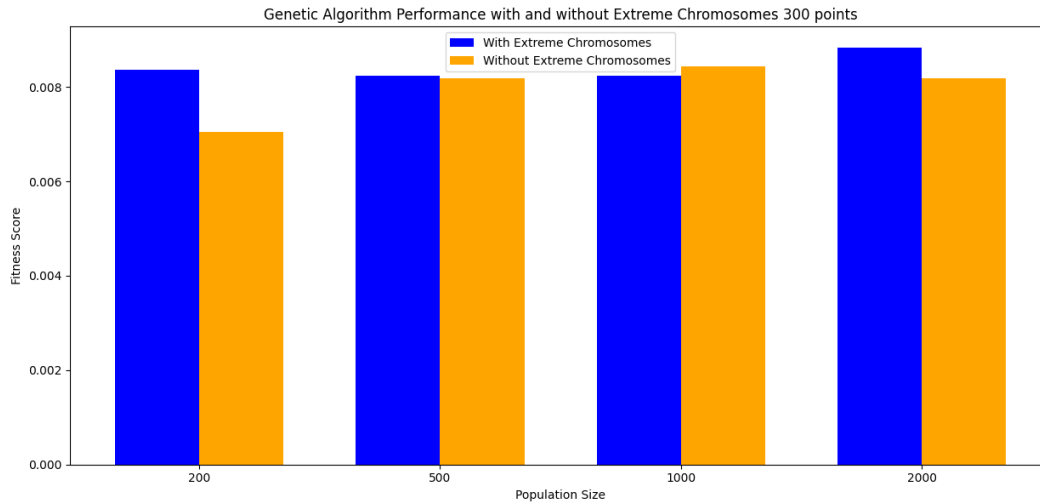


Fig. 15. Fitness score of the best route for different mutation methods with 300 points (charging stations).

1.4.6. Impact of Best Route Preservation

Preserving the top-performing chromosome from one generation to the next significantly enhanced the genetic algorithm's performance, especially with more complex scenarios like 300 charging stations. This strategy helped the algorithm maintain high-quality solutions over generations, improving its ability to explore the solution space without losing strong candidates during crossover or mutation. However, its effectiveness varied with the problem's complexity. For instance, with only 150 charging stations, the benefit was less pronounced and, at times, even led to reduced performance. This suggests that the success of best route preservation is influenced by factors like population size and problem complexity.

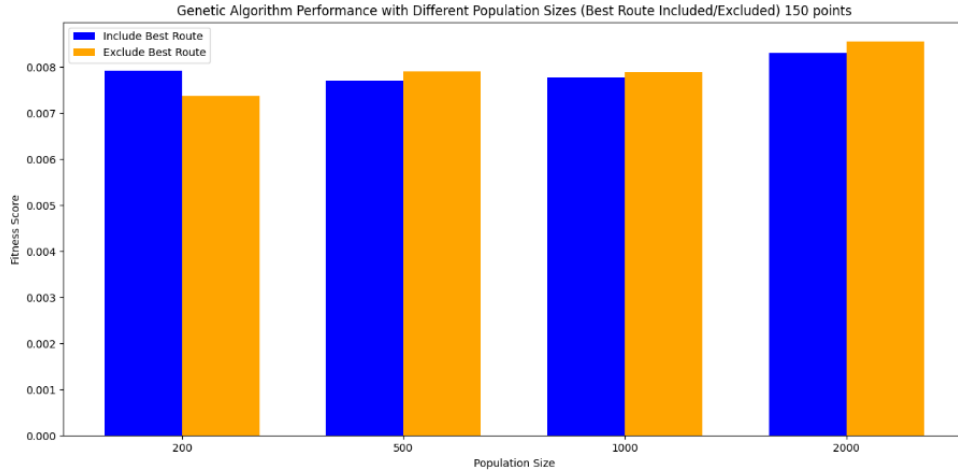


Fig. 16. Fitness score of the best route for with and with no Best Route Preservation with 150 points (charging stations).

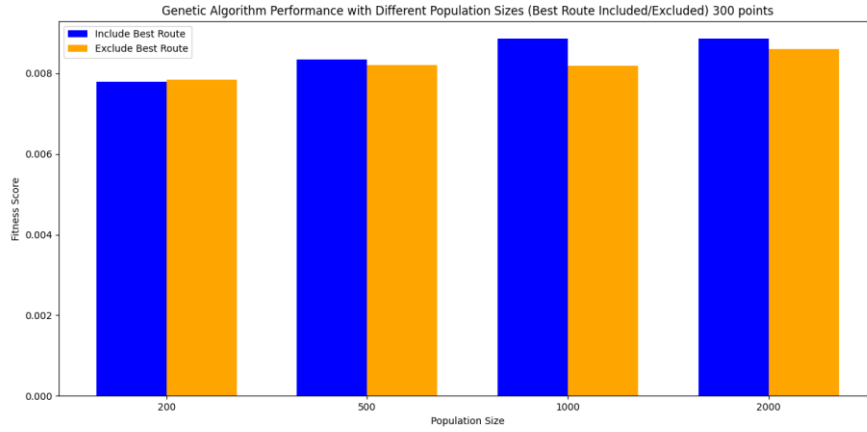


Fig. 17. Fitness score of the best route for with and with no Best Route Preservation with 300 points (charging stations).

1.4.7. Impact of Passing the Best Route During Dynamic Assessment

In the dynamic process when recalculating the best route for the updated segment, the top-performing route and associated charging station data from the current segment are passed forward to the next. This approach of carrying the best route into subsequent segments during dynamic assessment significantly impacts the algorithm's performance. For scenarios with 150 points, this strategy provides modest improvements across most population sizes. However, with more complex scenarios, like 300 points, the performance boost becomes more pronounced and consistent. This indicates that the effectiveness of passing the best route is closely linked to problem complexity, with larger and more complex instances benefiting more from this strategy.

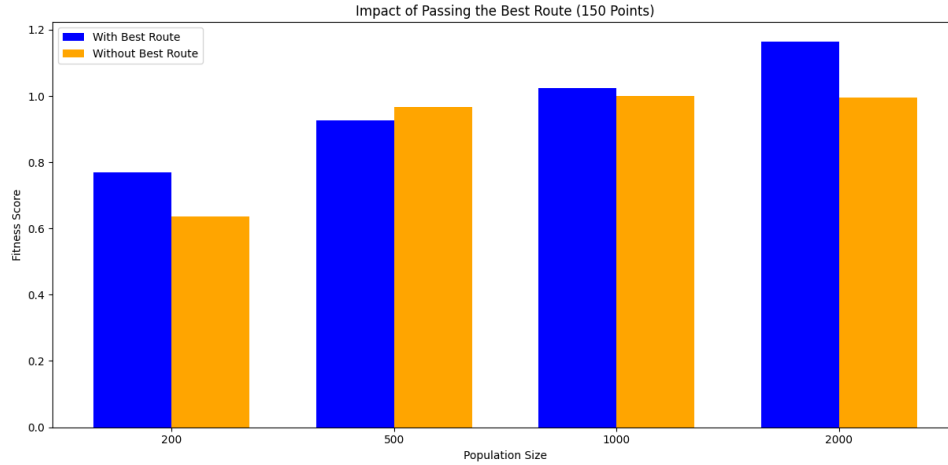


Fig. 18. Fitness score of the best route for with and with no Passing the Best Route During Dynamic Assessment with 150 points.

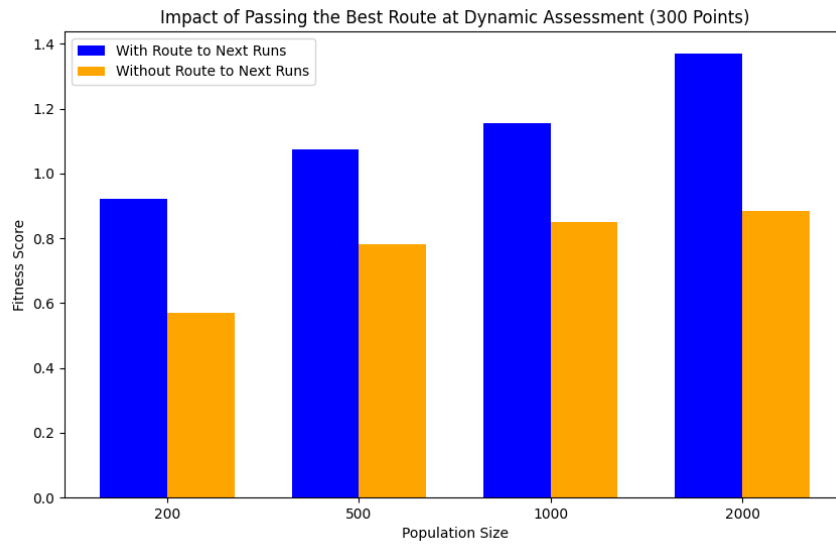


Fig. 19. Fitness score of the best route for with and with no Passing the Best Route During Dynamic Assessment with 300 points.

1.4.8. Conclusions from the Results

The study demonstrates that the parameters of the genetic algorithm significantly influence its efficiency, especially as the problem complexity increases. In simpler scenarios with fewer charging stations, the differences between methods are less noticeable. However, as the complexity grows, the impact of these parameters becomes more pronounced. Tournament Selection, Variant Crossover, and Variant Mutation showed superior performance, particularly in handling complex cases. Preserving the best route for future segments proved highly beneficial in complex scenarios but had a less consistent impact in simpler cases. Overall, the combination of the genetic algorithm with dynamic route reassessment and queuing time simulation provided a robust and efficient solution for optimizing EV routes, effectively balancing exploration and exploitation across varying levels of complexity.

2. User Guide

2.1. Overview of the System

The system is designed to dynamically generate an optimal route for an electric vehicle (EV), considering key parameters like battery capacity, route, and charging stations. The route is optimized to ensure efficient travel by selecting the most suitable charging stations along the way. The system uses a combination of pre-defined checkpoints, route generation algorithms, and simulated conditions (like queueing times at charging stations) to make real-time decisions.

2.2. Key parameters configuration (From parameters.py)

EV_CAPACITY: This parameter represents the electric vehicle's battery capacity. It dictates the distance the EV can travel before needing to recharge, directly impacting how often charging stations are considered during the route.

NUM_ROUTE_POINTS: This parameter defines the number of route points along the EV's journey and, accordingly, the number of route segments. Having more route points allows for finer adjustments, making the route more responsive to dynamic changes, such as available charging stations.

Population Size: This parameter refers to the number of candidate routes considered in each iteration of the optimization process. A higher population size means a larger search space, which increases the diversity of potential solutions and improves the chances of finding an optimal route.

Generations: This parameter defines how many times the algorithm will perform the reproduction process, which includes selecting parents, crossover, and mutation. A greater number of generations allows for more iterations of refinement, potentially leading to a more optimal route over time.

MUTATION_RATE: This controls the rate at which random changes (mutations) are introduced to the candidate routes during the optimization process. It helps ensure the algorithm explores alternative routes.

NUM_POINTS: This parameter defines the number of optional charging stations to choose from along the route. A higher number of charging station points provides more opportunities to optimize the best route by offering more choices for charging stops, but it also increases the complexity of the optimization process.

MAX_STAGNATION: This parameter represents the maximum number of generations allowed without any improvement in the route. When stagnation occurs, it means the algorithm has stopped finding better routes. Once this limit is reached, the algorithm terminates, taking the best route found so far.

CHECK_POINTS: Pre-defined key locations the route passes through, to make the route more complicated and complex.

AVERAGE_QUEUEING_TIME: Average waiting time (in minutes) at charging stations.

MIN_QUEUEING_TIME and MAX_QUEUEING_TIME: These parameters define the range of optional waiting times at charging stations. `MIN_QUEUEING_TIME` represents the minimum value, or the shortest possible simulated wait time, while `MAX_QUEUEING_TIME` represents the maximum value or the longest allowable wait time at any station in the simulation.

By adjusting these parameters, users can examine how effectively the genetic algorithm handles route optimization, the complexity of the problem, and the level of resources they wish to invest in finding a solution in terms of computational power and runtime. Adjusting these values allows users to fine-tune the algorithm to their specific needs or experiment with different configurations.

2.3. Visual and execution output:

In addition to the visual output provided in section 1.3.3 (Visualization of the Dynamic Process), the algorithm provides valuable performance metrics, such as:

- Total execution time (in seconds).
- Best final route: The ID numbers of the charging stations chosen by the algorithm for the EV to stop and recharge. For example, in Figure 20, the selected charging stations are 8, 31, 43, 72, 97, and 129, which represent the locations where the EV must stop to complete the route.
- Final Fitness Score: A measure of route efficiency, a higher fitness score represents a better route. For example, in Figure 20, the fitness score is 0.865742.

```
Total execution time: 20.42 seconds
Best final route: [8, 31, 43, 72, 97, 129]
Final Fitness Score: 0.865742
```

Fig. 20. Execution output.

2.4. How the algorithm generates the route and user customization:

The algorithm uses a series of steps to generate a dynamic route, which can be customized based on the user's requirements:

Route Generation Steps:

1. **Initial Setup:** The `generate_initial_data()` function creates a route with checkpoints using `generate_route_with_checkpoints()`.
2. **Checkpoints:** Key locations (`CHECK_POINTS`) are used to segment the route.

3. **Waypoints:** Additional waypoints between checkpoints are generated with some random variations for added realism.
4. **Charging Stations:** Random charging stations with varying queue times are added along the route.

Customization Options:

- **Route Points:** Adjust `NUM_ROUTE_POINTS` to change the number of points and details in the route.
- **Checkpoints:** Modify the `CHECK_POINTS` array to set specific key locations along the route.
- **Zigzags:** Change `turn_amplitude` to control the degree of random turns in the route.
- **Charging Stations:** Customize the number of charging stations with `NUM_POINTS` and control randomness with seed values.

This level of customization allows users to define routes that best fit their needs, be it for real-world EV charging optimization or route planning experiments.

3. Maintenance Guide

3.1. Purpose

This guide ensures the continued use and maintenance of the project after completion. It provides instructions for making changes, updates, and improvements, managing the software environment, customizing components, and keeping the system flexible for various needs.

3.2. Operating Environment

3.2.1. Software Requirements

- **Python Version:** The system is compatible with Python 3.x. Ensure that you have Python 3.7 or later installed on your machine.
- **Package Dependencies:** The following Python packages are required for the system to function properly:
 - `numpy`
 - `matplotlib`
 - `scikit-learn`
 - `copy`
 - `os`
 - `time`

You can install all required dependencies using the provided `requirements.txt` file with the following command:

```
pip install -r requirements.txt
```

3.2.2. Installation Instructions

To install and run the system:

1. Clone the repository: Download the code from the GitHub repository using the following command in Command Prompt (CMD):

```
git clone https://github.com/Yovelgabay/Electric-Vehicle-Routing.git
```

2. Navigate to the project directory:

```
cd Electric-Vehicle-Routing
```

3. Install dependencies: Install all required Python packages as described above.

```
pip install -r requirements.txt
```

4. Run the program: Execute the main script to generate routes and perform the optimization.

```
python main.py
```

3.3. Flexibility and Customization

The code is designed to be flexible, allowing users to experiment with different configurations and combinations of the implemented methods. The system supports the integration of various genetic algorithm operators, including crossover, mutation, and selection methods, to suit diverse research purposes or real-world applications. To explore all different operator combinations, users can run the **main_tests** and **ga_tests** scripts. These files contain the necessary tests for evaluating the different crossover, mutation, and selection methods, allowing users to optimize and customize the system's performance based on their specific requirements.

3.3.1. Flexible Combinations of Methods

The system allows for choosing different combinations among the following methods:

- Crossover: Users can switch between different crossover methods, such as Variant Crossover, Double-Point Crossover, and Uniform Crossover, depending on the desired level of diversity in the population and the problem complexity.
- Mutation: Users can choose between the Variant Mutation method or Random Resetting Mutation, depending on the variability they want to introduce during the algorithm's execution.
- Selection: Different parent selection methods like Tournament Selection, Roulette Wheel Selection, and Rank Selection can be chosen to control how parent chromosomes are selected for reproduction.

3.3.2. Adding New Methods

The system's modular architecture allows for the addition of new selection, mutation, or crossover methods without significant structural changes. To add a new method:

1. Create a new function for the new method inside the appropriate module (e.g., `ga_tests.py`).
2. Modify the main genetic algorithm loop to call the newly created method instead of the existing one.

4. References

1. Liu, Fei, Chengyu Lu, Lin Gui, Qingfu Zhang, Xialiang Tong, and Mingxuan Yuan. "Heuristics for vehicle routing problem: A survey and recent advances." arXiv preprint arXiv:2303.04147 (2023). <https://arxiv.org/pdf/2303.04147>
2. Dantzig, George B., and John H. Ramser. "The truck dispatching problem." *Management Science* 6, no. 1 (1959): 80-91. <https://pubsonline.informs.org/doi/10.1287/mnsc.6.1.80>
3. Gintaras Vaira. "GENETIC ALGORITHM FOR VEHICLE ROUTING PROBLEM". Doctoral Dissertation Technological Sciences, Informatics Engineering (07 T) (2014). https://www.mii.lt/files/doc/lt/doktorantura/apgintos_disertacijos/mii_dis_2014_vaira.pdf
4. IEA. "Fast Facts on Transportation Greenhouse Gas Emissions" (2023). <https://www.epa.gov/greenvehicles/fast-facts-transportation-greenhouse-gas-emissions>
5. Ilker Kucukoglu, Reginald Dewil and Dirk Cattrysse. "The electric vehicle routing problem and its variations: A literature review". *Expert Systems with Applications* Volume 161, 107650 (2021). <https://www.sciencedirect.com/science/article/abs/pii/S0360835221005544?via%3DiHub>
6. Merve Keskin, Gilbert Laporte and Bülent Çatay. "Electric Vehicle Routing Problem with Time-Dependent Waiting Times at Recharging Stations". *Computers & Operations Research* Volume 107, Pages 77-94 (2019). <https://www.sciencedirect.com/science/article/abs/pii/S030505481930053X>
7. Chang Wook Ahn, R. S. Ramakrishna. "A Genetic Algorithm for Shortest Path Routing Problem and the Sizing of Populations" (2022). https://www.academia.edu/30609973/A_Genetic_Algorithm_for_Shortest_Path_Routing_Problem_and_the_Sizing_of_Populations
8. Brad L. Miller and David E. Goldberg. "Genetic Algorithms, Tournament Selection, and the Effects of Noise". *Complex Systems* 9, 193- 2 (1995). <https://wpmedia.wolfram.com/sites/13/2018/02/09-3-2.pdf>
9. Seyedali Mirjalili. "Evolutionary Algorithms and Neural Networks". Pages 43-55 (2019). <https://link.springer.com/book/10.1007/978-3-319-93025-1>
10. L. Haldurai, T. Madhubala and R. Rajalakshmi. "A Study on Genetic Algorithm and its Applications". *International Journal of Computer Sciences and Engineering* Volume-4, Issue-10 (2016). https://www.ijcseonline.org/pub_paper/22-IJCSE-01860-4.pdf

11. L. Haldurai, T. Madhubala and R. Rajalakshmi. "A Study on Genetic Algorithm and its Applications". International Journal of Computer Sciences and Engineering Volume-4, Issue-10 (2016). https://www.ijcseonline.org/pub_paper/22-IJCSE-01860-4.pdf